PowerGlove Vision

POWERGLOVE VISION / PROJECT DOCUMENTATION

Camera-only hand tracking for an Arduino UNO Q and a RetroPie arcade cabinet. The hand may be bare or covered by a plain white or black glove; the glove is only a visual tracking aid and contains no electronics.

The project has two processes:

- `powerglove-vision` runs on the UNO Q. It observes one hand, calibrates a comfortable neutral pose, converts motion and finger flex into controller state, and sends that state over UDP.
- `powerglove-receiver` runs on the Raspberry Pi. It exposes the received state as a Linux virtual gamepad that RetroArch can bind like any other controller.

The current milestone supplies a complete standard-gamepad path, all nine cartridge-free Programs A-I, automatic RetroPie game selection, and preserves four analogue channels plus individual finger values in the network protocol. Native Power Glove packets for Super Glove Ball will be added later through an `lr-nestopia` integration; no ROM modification is required.

Bad Street Brawler's unusual cartridge-resident Programs A-I and their direct camera equivalents are documented in `docs/bad-street-brawler-programs.md`.

Controls

Eleven profiles are included: Programs A-I plus dedicated mappings for Bad Street Brawler and Super Glove Ball.

Programs A-I

These profiles reproduce the useful controller output of the original glove programs directly. Bad Street Brawler never needs to be started. The included game registry selects B for Joust, C for Gyruss, E for Defender II, F for Sesame Street 1-2-3, G for Gun.Smoke, and I for Knight Rider. Programs A, D, and H are ready to assign to any ROM in `config/games.json`.

Bad Street Brawler

Motion	Output
Move hand left or right	D-pad left or right
Raise or lower hand	D-pad up or down
Curl thumb	Pulsed NES B
Curl middle finger	NES A+B
Roll hand clockwise/counter-clockwise	A+right / A+left
Push hand toward the camera	Glove Zap auxiliary event
Hold a V sign	Start
Hold a thumbs-up with other fingers closed	Select

The auxiliary event is exposed as `BTN_TR2`. Standard `lr-fceumm` cannot unlock the game's glove-only zap from an ordinary controller; the event is retained so the future native-glove core can consume it.

Super Glove Ball

X, Y, estimated depth, wrist roll, and all five finger curl values are sent continuously. The standard-gamepad fallback maps hand position to the D-pad, index curl to A, and thumb curl to B.

UNO Q setup

Use the 4 GB UNO Q when possible. Connect a UVC USB camera through an externally powered USB-C hub, then connect to the board over the network.

SH

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e '.[vision]'
```

Run the tracker, replacing the receiver address and token:

SH

```
powerglove-vision \
  --receiver 192.168.1.50 \
  --token 'replace-with-a-long-random-value' \
  --profile bad_street_brawler \
  --glove-color white
```

The vision service also listens for authenticated profile changes from RetroPie on UDP port 55356. A change first releases every virtual control, starts a new packet session, changes the mapping, begins neutral-position calibration, and acknowledges the selected profile.

Open `http://<uno-q-address>:8088` from another machine. The page shows the camera overlay, current output, tracking confidence, and a **Center hand** button. Calibration also begins automatically at startup.

Start and Select use poses held for about 0.7 seconds, then emit a short button pulse. While either menu pose is forming, ordinary attack controls are suppressed so curled fingers cannot fire an unwanted move.

For a black glove, use a light, uncluttered background. For a white glove, use a dark background. Landmark detection remains the primary tracker in both cases; contrast is more useful than a specific colour.

UNO Q blue matrix

The optional App Lab sketch in `uno_q/sketch/` drives the built-in 8x13 blue matrix. It deliberately does not replace the UNO Q's protected early system boot display. Once the PowerGlove Vision app starts, it shows:

- an animated, scanning 8-bit hand while camera and model components load;
- a blocky `PG` emblem before a game profile is selected;
- a large `A`-`I`, `BS`, or `GB` acknowledgement for the active profile;
- a gently pulsing version of that acknowledgement while tracking is active;
- a blinking X if camera or runtime initialization fails.

Copy `uno_q/sketch/` into the sketch portion of the PowerGlove Vision App Lab app. The Python process detects App Lab's Bridge automatically and calls the sketch; on a development computer it quietly runs without matrix support. Use `--no-matrix` to disable the integration explicitly.

In App Lab, enable **Run at startup** for the completed custom app. The Arduino system boot logo remains visible during Linux startup, after which the custom loading and ready states take over.

Raspberry Pi receiver

The receiver requires Python `evdev` and access to `/dev/uinput`:

SH

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -e '.[receiver]'
powerglove-receiver \
  --listen 0.0.0.0 \
  --token 'replace-with-the-same-long-random-value'
```

Use `--dry-run` first to inspect incoming state without creating a device. The receiver releases every control if packets stop for 250 ms, so a lost camera, network interruption, or stopped UNO Q cannot leave a direction held.

After the virtual device named `PowerGlove Vision` appears, bind it in RetroArch as Player 1 or add it as another input source to the cabinet's existing gamepad merger.

Automatic RetroPie profiles

Install this package in `/opt/powerglove`, then copy:

- `config/games.json` to `/etc/powerglove/games.json`;
- `config/launcher.example.json` to `/etc/powerglove/launcher.json` and set the UNO Q address;
- the same long random token used by the UNO Q to `/etc/powerglove/token`.

The project provides `powerglove-retropie-hook`. Call the supplied `retropie/runcommand-onstart-powerglove.sh` from the cabinet's existing `runcommand-onstart.sh`, forwarding its four arguments. Call the supplied end script from `runcommand-onend.sh`. Do not replace the existing cabinet hooks; they also manage controller order, RGB processes, and trackballs.

The start hook uses the exact ROM basename. Known NES games receive their configured profile; other games and systems explicitly turn gestures off. The end hook also turns gestures off. Communication failure is logged but never prevents a game from launching or interferes with conventional controls.

To add a game, add its exact ROM filename and one of `program_a` through `program_i`, `bad_street_brawler`, or `super_glove_ball` to the `games` object. The supplied registry recognizes `.nes`, `.zip`, and `.7z` copies of the two provided glove games. A temporary manual override is also available:

SH

```
powerglove-profile --uno-q 192.168.1.60 --token-file /etc/powerglove/token \
  --profile program_a
```

Tuning

Start with the defaults in `config/profiles.json`. Important settings are:

- `move_on` and `move_off`: directional activation and release thresholds.
- `curl_on` and `curl_off`: finger hysteresis.
- `roll_on` and `roll_off`: wrist-roll thresholds in normalized quarter-turns.
- `push_on` and `push_off`: apparent-hand-size change used for monocular depth.
- `pulse_hz`: Bad Street Brawler's pulsed B rate.

Use the debug page while adjusting thresholds. Keep `*_off` lower than `*_on` to prevent controls chattering near a boundary.

Development

Core tests do not require a camera or MediaPipe:

SH

```
python -m unittest discover -s tests -v
```

The code intentionally separates observations, gesture mapping, transport, and Linux input output.
Recorded-camera replay and a native Nestopia adapter can be added without changing the gesture engine.